

Classic Network Model Analysis

CHEN PENG
SCHOOL OF ARTIFICIAL INTELLIGENCE
HEBEI UNIVERSITY OF TECHNOLOGY



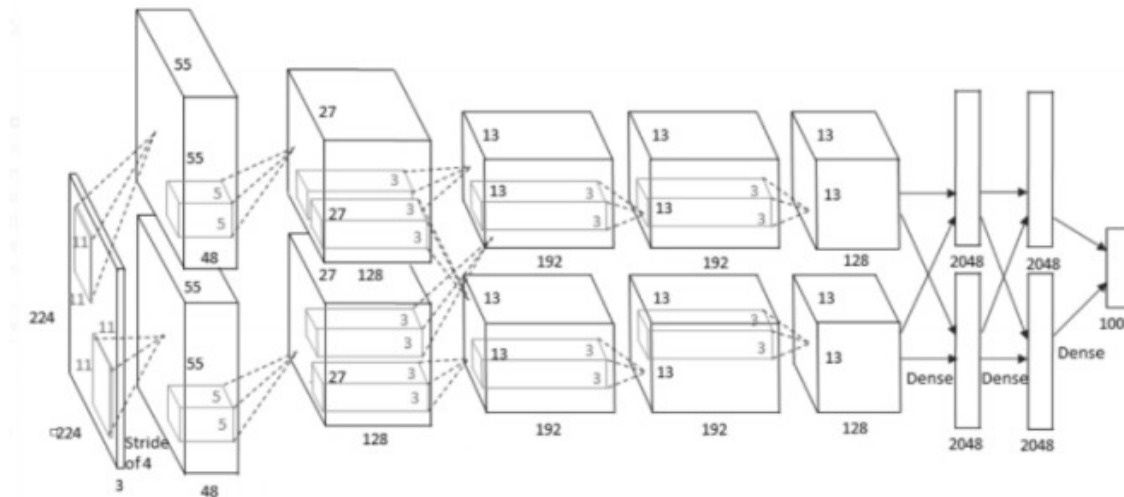
Classic Networks

- AlexNet
- ZFNet
- VGG
- GoogLeNet
- ResNet

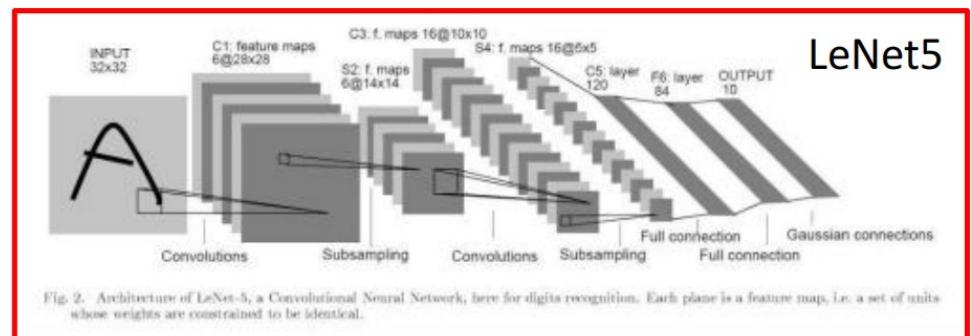


AlexNet

AlexNet



AlexNet—2012 ImageNet Large Scale Visual Recognition Challenge champion, improving accuracy by almost 10 percentage points!



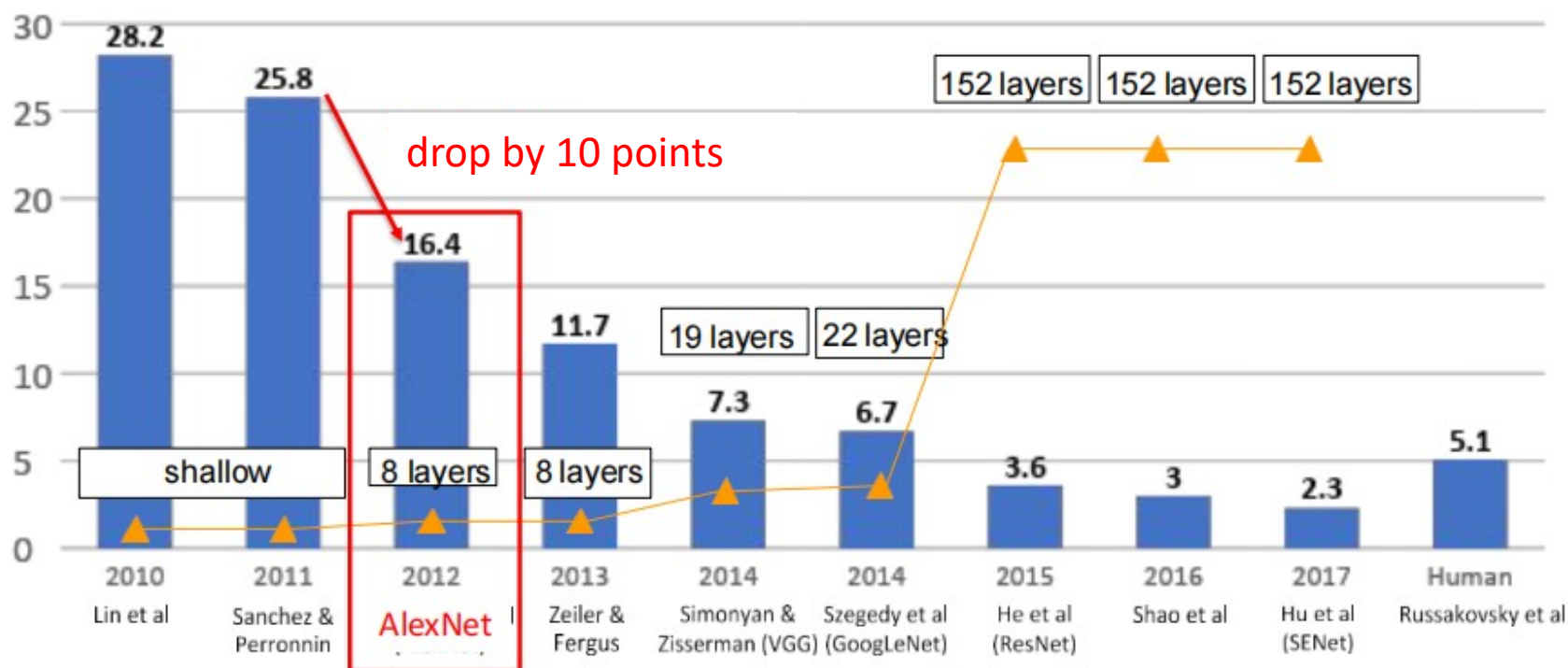
ImageNet Large Scale Visual Recognition Challenge

ImageNet Large Scale Visual Recognition Challenge (ILSVRC)

- One of the most prestigious academic competitions in the field of computer vision.
- ImageNet Dataset — led by Professor Fei-Fei Li of Stanford University, it contains over 14 million full-resolution labeled images.
- ILSVRC extracts subsets from the ImageNet dataset as competition data.

2012 ILSVRC — 1,281,167 training images, 50,000 validation images, 100,000 test images.

ImageNet competition champions over the years

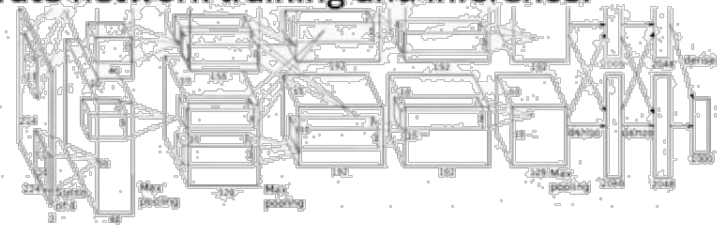


Classic Network Analysis—AlexNet

✿ AlexNet——Validated the efficiency of deep convolutional neural networks

Main Contributions

1. Proposed a convolutional neural network structure combining convolutional layers and fully connected
2. For the first time, used the ReLU function as the activation function for the neural network;
3. For the first time, proposed Dropout regularization to control overfitting;
4. Used a mini-batch gradient descent algorithm with momentum to accelerate training convergence;
5. Used data augmentation strategies to greatly suppress overfitting during training;
6. Utilized the parallel computing power of GPUs to accelerate network training and inference.



AlexNet

Convolutional Layer

Structure:

CONV1

MAX POOL1

NORM1

CONV2

MAX POOL2

NORM2

CONV3

CONV4

CONV5

Max POOL3

FC6

FC7

FC8

Pooling Layer

Structure:

CONV1

MAX POOL1

NORM1

CONV2

MAX POOL2

NORM2

CONV3

CONV4

CONV5

Max POOL3

FC6

FC7

FC8

Local Response Normalization Layer

Structure:

CONV1

MAX POOL1

NORM1

CONV2

MAX POOL2

NORM2

CONV3

CONV4

CONV5

Max POOL3

FC6

FC7

FC8

Fully Connected Layer

Structure:

CONV1

MAX POOL1

NORM1

CONV2

MAX POOL2

NORM2

CONV3

CONV4

CONV5

Max POOL3

FC6

FC7

FC8

Classic Network Analysis—AlexNet

Structure:

CONV1
MAX POOL1
NORM1
CONV2
MAX POOL2
NORM2
CONV3
CONV4
CONV5
Max POOL3
FC6
FC7
FC8

Layer Count Statistic Explanation:

- When calculating the number of network layers, only convolutional layers and fully connected layers are counted;
- Pooling layers and various normalization layers are post-processing on the preceding convolutional layer output feature maps, and are not counted as separate layers.

AlexNet has a total of 8 layers:

- 5 convolutional layers (CONV1—CONV5)
- 3 fully connected layers (FC6–FC8)

Classical Network Analysis——AlexNet

结构:

CONV1

MAX POOL1

NORM1

CONV2

MAX POOL2

NORM2

CONV3

CONV4

CONV5

Max POOL3

FC6

FC7

FC8

First Layer (CONV1): 96 11x11 convolutional kernels, with a stride of 4, no zero padding

Question: Input: 227x227x3 size image, what is the number and dimensions of the output feature maps?

Dimensions: $(227-11) / 4 + 1 = 55$

Number: 96

Question: How many parameters does this layer have?

Parameters: $(11*11*3+1) * 96 = 35K$

➤ The first convolutional layer extracted response information for 96 kinds of structures, obtaining 96 characteristic response maps;

➤ Each element of the feature map is output after processing by the ReLU function;

➤ **INPUT ZERO-CENTERED MEAN NORMALIZATION**

Classic Network Analysis—AlexNet

Structure:

CONV1

MAX POOL1

NORM1

CONV2

MAX POOL2

NORM2

CONV3

CONV4

CONV5

Max POOL3

FC6

FC7

FC8

Max POOL1: Window size 3x3, stride 2

Overlapping helps
combat overfitting

Function: Reduce feature map dimensions, counter the effects of slight object shifts

Output dimensions: $(55-3) / 2 + 1 = 27$

Number of feature maps: 96

Number of parameters: 0

Classical Network Analysis—AlexNet

Structure:

CONV1

MAX POOL1

NORM1

CONV2

MAX POOL2

NORM2

CONV3

CONV4

CONV5

Max POOL3

FC6

FC7

FC8

Local Response

Normalization Layer (NORM1)



- Creates a competitive mechanism for the activities of local neurons;
- Cells with larger responses become relatively larger;
- Suppresses other neurons with smaller feedback;
- Enhances the generalization ability of the model.

Later research indicated:

In deeper networks, this layer's contribution to improving classification performance is not significant, and it will increase computational load and storage space.

Classic Network Analysis—AlexNet

Structure

CONV1

MAX POOL1

NORM1

CONV2

MAX POOL2

NORM2

CONV3

CONV4

CONV5

Max POOL3

FC6

FC7

FC8

Second Layer (CONV2): 256 5x5 convolution kernels, stride of 1, ue-padding $p=2$

Question: For an input feature map block of 8, what are the number and size?

Size: $(27 - 5 + 2*2) / 1+1 = 27$

Number: 256

Receptive field expanded nearly 10 times

Classic Network Analysis—AlexNet

CONV1

MAX POOL1

NORM1

CONV2

MAX POOL2

NORM2

CONV3

CONV4

CONV5

Max POOL3

FC6

FC7

FC8

Second Layer (CONV2): 256 5x5 convolution kernels, stride of 1, ue-padding p=2

Question: For an input feature map block of 8, what are the number and size?

Size: $(27 - 5 + 2*2) / 1+1 = 27$

Number: 256

Classic Network Analysis—AlexNet

Structure:

CONV1

MAX POOL1

NORM1

CONV2

MAX POOL2

NORM2

CONV3

CONV4

CONV5

Max POOL3

FC6

FC7

FC8

Third, Fourth Layers (CONV3, CONV4): 384 3x3 conv kernels, stride 1, use zero-padding $p=1$

Question: For CONV3 input: 13x13x256 sized feature map group, what are the number and size of output feature maps?

Size: $(13 - 3 + 2*1)/1+1 = 13$

Number: 384

No maximum pooling or local response normalization performed

Classical Network Analysis—AlexNet

Structure

CONV1

MAX POOL1

NORM1

CONV2

MAX POOL2

NORM2

CONV3

CONV4

CONV5

Max POOL3

FC6

FC7

FC8

Fifth Layer (CONV5): 256 3x3 convolutions, stride 1, use zero-padding=1

Use a max pooling layer to further reduce feature map dimensions

After the layer convolution, feature map size is $13 \times 13 \times 256$;

Max pooling uses a 3×3 kernel;

Max pooling, the feature map size has $6 \times 6 \times 256$

Classical Network Analysis——AlexNet

Structure:

CONV1

MAX POOL1

NORM1

CONV2

MAX POOL2

NORM2

CONV3

CONV4

CONV5

Max POOL3

FC6

FC7

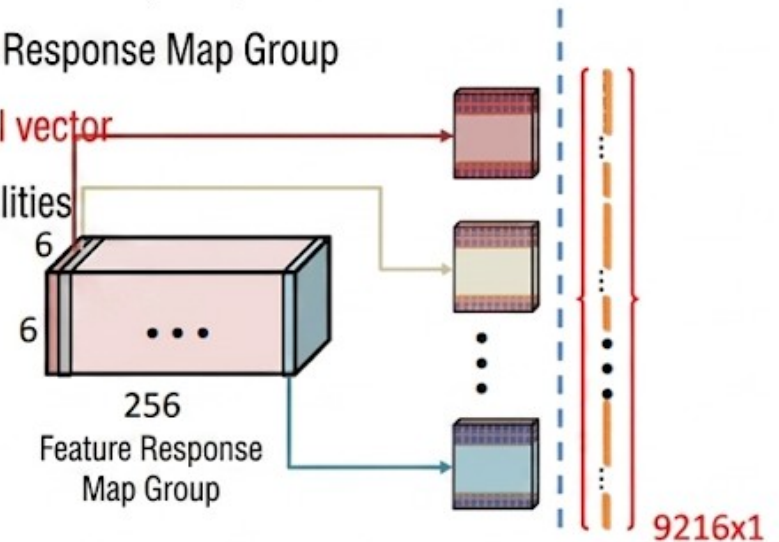
FC8

Sixth-Eighth Layers (FC6, FC7, FC8): Fully Connected Neural Network Classifier

Max POOL3 Output: Feature Response Map Group

FC6 Input: 9216-dimensional vector

Output: Image Class Probabilities



Classical Network Analysis——AlexNet

Structure:

CONV1
MAX POOL1
NORM1
CONV2
MAX POOL2
NORM2
CONV3
CONV4
CONV5
Max POOL3
FC6
FC7
FC8

Important Note:

- The convolutional layers for image feature extraction and the fully connected layers for classification are learned simultaneously;
- The convolutional layers and fully connected layers **influence and promote each other** in the learning process;

Classic Network Analysis—AlexNet

Structure:

CONV1

MAX POOL1

NORM1

CONV2

MAX POOL2

NORM2

CONV3

CONV4

CONV5

Max POOL3

FC6

FC7

FC8

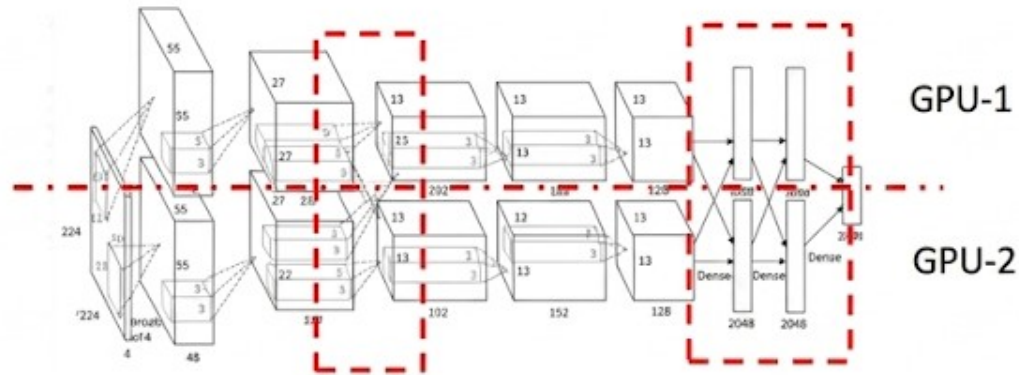
Important Techniques:

- Dropout strategy to prevent overfitting;
- Use stochastic gradient descent with momentum to accelerate convergence;
- Manually decrease the learning rate by 10x when the validation set loss does not decrease;
- Adopt data augmentation strategies to increase the number of training samples and prevent overfitting;
- **Ensemble multiple models to further improve accuracy.**

Classical Network Analysis—AlexNet

Structure:

CONV1
MAX POOL1
NORM1
CONV2
MAX POOL2
NORM2
CONV3
CONV4
CONV5
Max POOL3
FC6
FC7
FC8

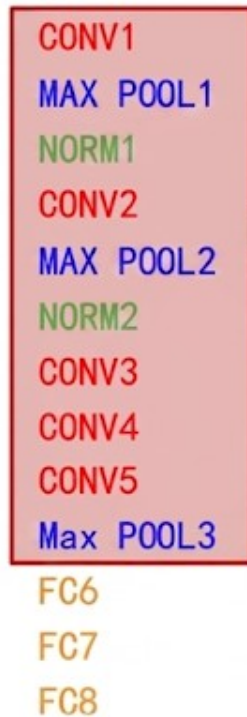


- AlexNet is distributed across two GPUs, each having half of the neurons;
- CONV1, CONV2, CONV4, and CONV5 only use feature maps output from the previous layer **within the same** GPU as input;
- CONV3, FC6, FC7, and FC8 concatenate *all* feature maps from the previous layer, implementing **cross-GPU computation** for the neural network.

GTX 580 GPU has *only* 3GB memory!!!

Question: What are AlexNet convolutional layers doing?

Input:
 $227 \times 227 \times 3$



Hyper-
kernel
Group



Output:
 $6 \times 6 \times 256$

Question: What are AlexNet convolutional layers doing?

Input:
227x227x3



CONV1

MAX POOL1

NORM1

CONV2

MAX POOL2

NORM2

CONV3

CONV4

CONV5

Max POOL3

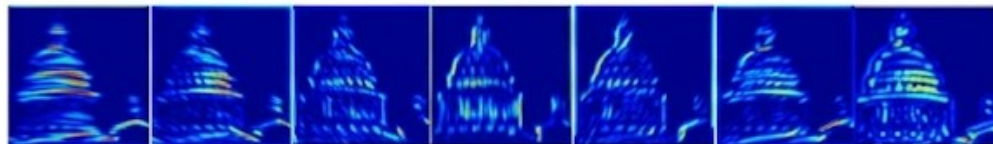
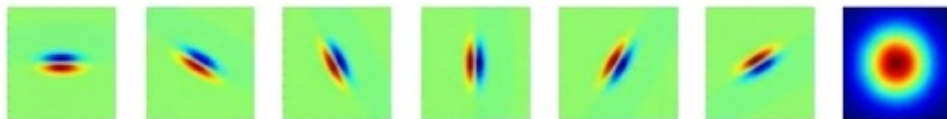
FC6

FC7

FC8



Output:
6x6x256



Question: What are AlexNet convolutional layers doing?

Input:
227x227x3

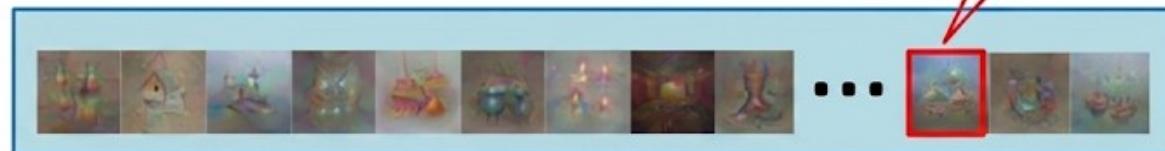


CONV1
MAX POOL1
NORM1
CONV2
MAX POOL2
NORM2
CONV3
CONV4
CONV5
Max POOL3

FC6
FC7
FC8



Output:
6x6x256



Super-kernel group



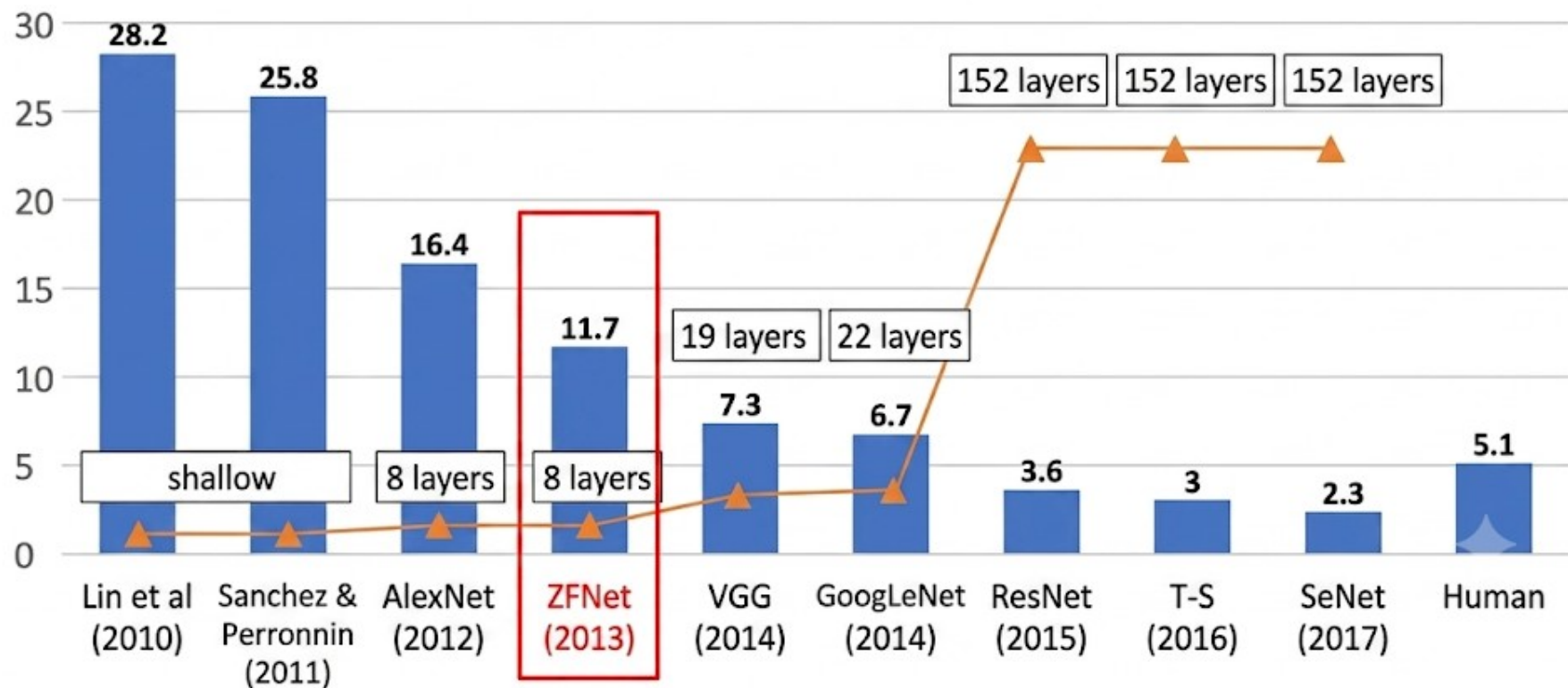
Feature map

High response



ZFNet

ImageNet Large Scale Visual Recognition Challenge Winners Over the Years



Classic Network Analysis — ZFNet

ZFNet structure:

CONV1

MAX POOL1

NORM1

CONV2

MAX POOL2

NORM2

CONV3

CONV4

CONV5

Max POOL3

FC6

FC7

FC8

Basically consistent with AlexNet structure!

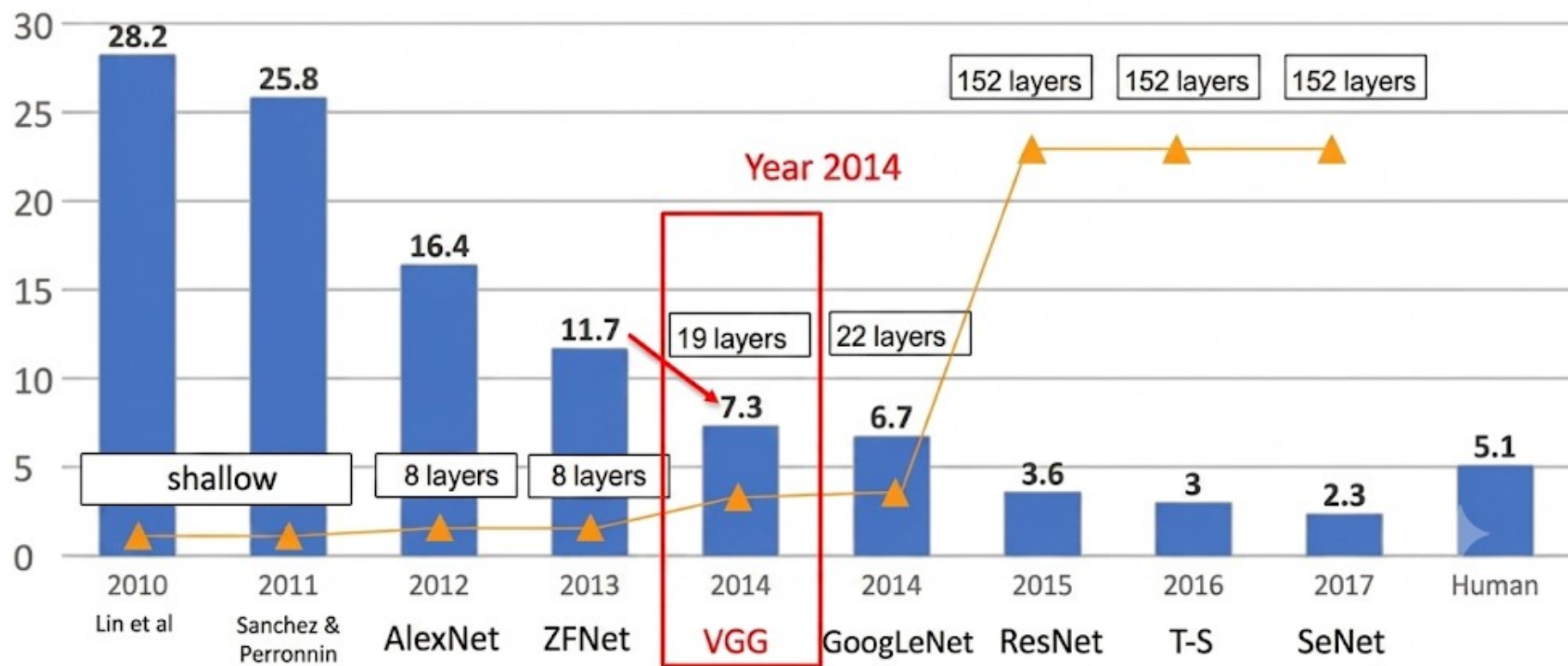
Key improvements:

- Changed the first convutial layer's kernel size to 7x7;
- Set second and thiutional layers the stride of stride to 2;
- Increased the third and foolvutional number of kernels.



VGG

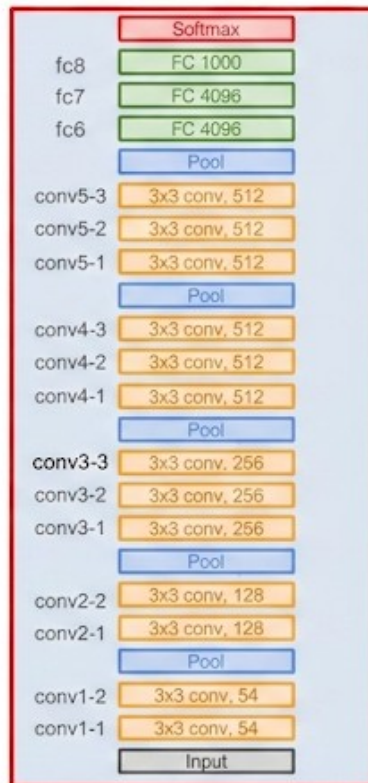
ImageNet Challenge Winners Over the Years



Classic Network Analysis — VGG



AlexNet

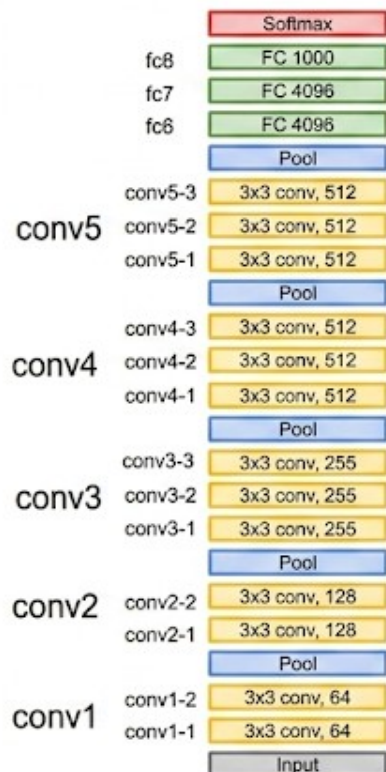


VGG16

VGG Network Contributions:

- Use smaller 3x3 convolutional kernels in succession to obtain a larger receptive field;
- Abandon the use of large kernels like 11x11 and 5x5;
- Deeper, stronger non-linearity, and fewer network parameters;
- Removed the Local Response Normalization (LRN) layer from AlexNet.
- Changed the way input image is mean-normalized.
- Structurally, it consists of 5 groups of convolutional layers + 3 fully connected layers.

Classic Network Analysis — VGG



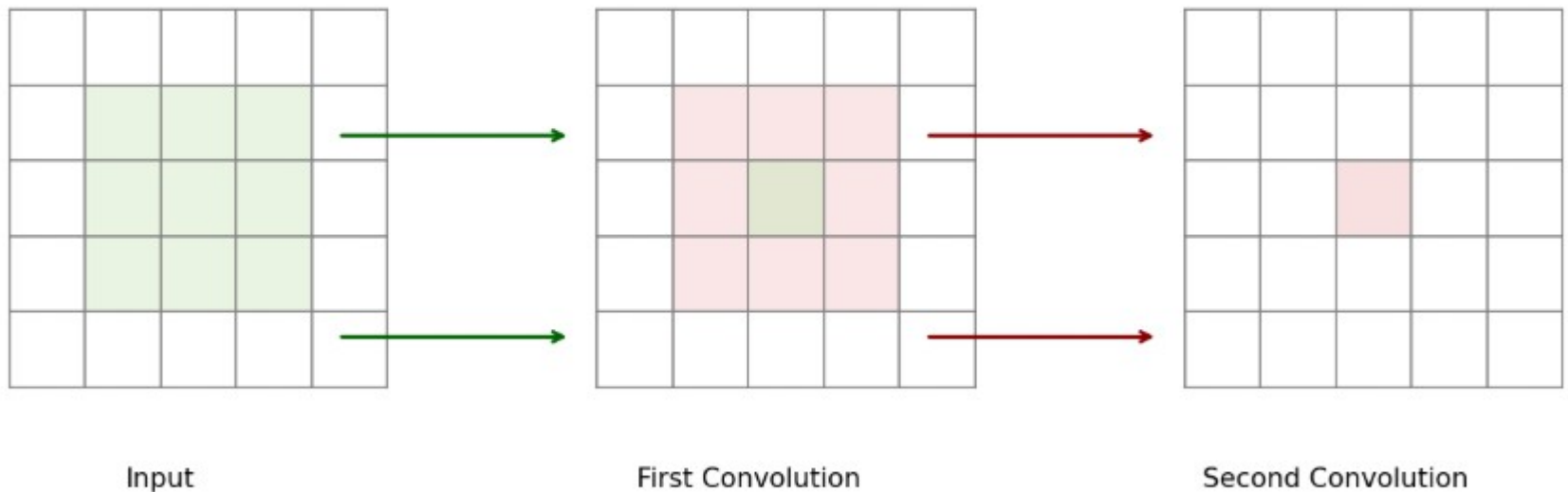
VGG16

- 13 convolutional layers and 3 fully connected layers
- Divided into 5 conv1, ..., conv5, in each stage's convolutional layers
- All convolutional layers use 3x3 convolutions and ReLU activation;
- All pooling layers use max pooling, ReLU activation;
- All pooling layers use max pooling, window size of 2x2, stride of 2;
- After each pooling operation, the number of convolutional kernels doubles, until reaching 512;
- Fully connected in Dropout strategy

Thinking

Question 1: What are the advantages of small convolution kernels?

Answer: Stacking multiple small-size convolution kernels in series can achieve the same receptive field as a large-size convolution kernel;



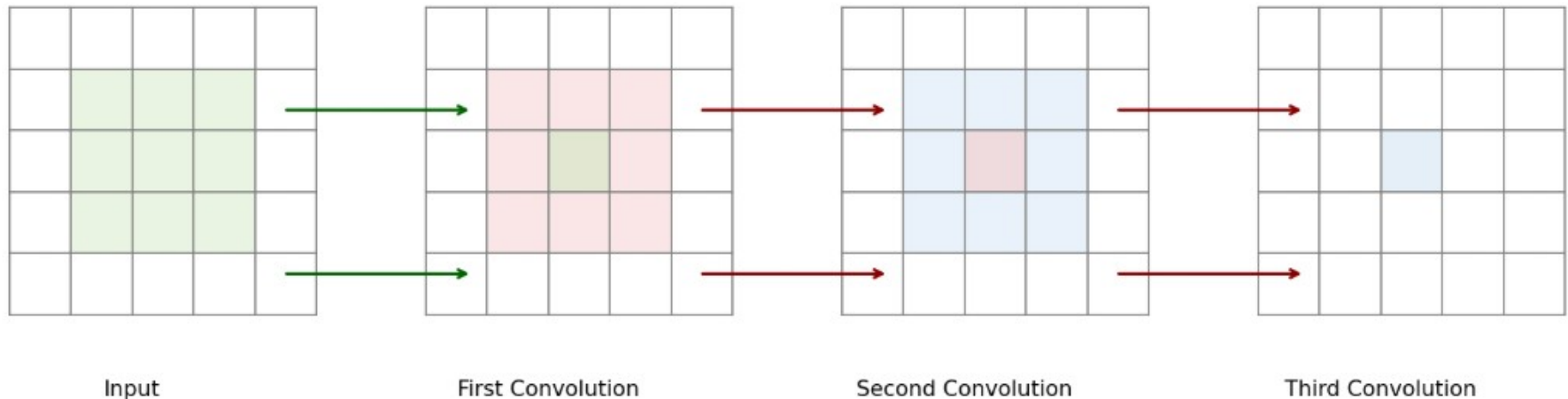
Two stacked 3x3 convolution kernels, receptive field = 5x5

Thinking

Question 1: What are the advantages of small convolution kernels?

Answer: Stacking multiple small-size convolution kernels in series can achieve the same receptive field as a large-size convolution kernel;

Networks built with stacked small kernels are deeper, more non-linear, and have fewer parameters.



Three stacked 3x3 convolution kernels, receptive field = 7x7

Thinking

Question 1: What are the advantages of small convolution kernels?

Answer: Stacking multiple small-size convolution kernels in series can achieve the same receptive field as a large-size convolution kernel;

Networks built with stacked small kernels are deeper, more non-linear, and have fewer parameters.

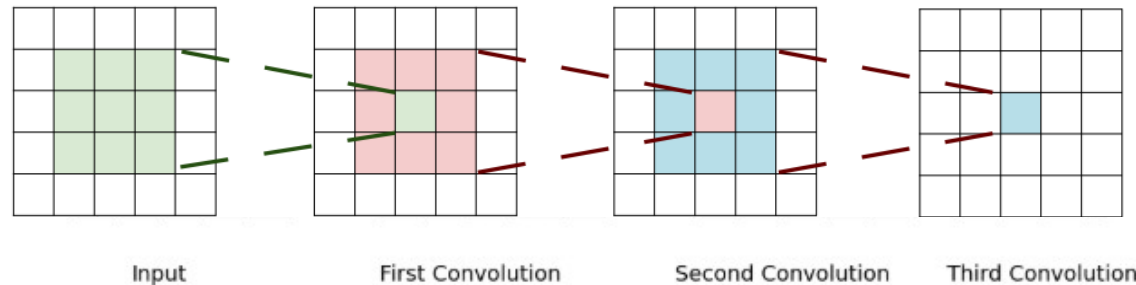
Assume the number of input/output channels of every conv layer is C :

Parameter count of three 3×3 conv layers stacked:

$$(3 \times 3 \times C) \times C \times 3 = 27C^2$$

Parameter count of a single 7×7 conv layer:

$$(7 \times 7 \times C) \times C = 49C^2$$



Three stacked 3×3 convolution kernels, receptive field = 7×7

Thinking

Question 2: Why does the number of convolution kernels double after each pooling operation in the first four stages of the VGG network?

Answer:

1. Pooling reduces the feature map size, lowering memory consumption.
2. Increasing the number of kernels helps learn richer structural features, but raises both the parameter count and memory usage.
3. This "halve the spatial size, double the channels" design balances recognition accuracy against storage and computation cost.

Ultimately, this boosts the overall network performance!

Thinking

Question 3: Why does the number of convolution kernels stop increasing once it reaches 512?

Answer:

1. The first fully-connected (FC) layer alone contains 102M parameters, accounting for 74% of the total parameters;
2. The parameter count of this FC layer equals the product of the feature map size and the number of channels;
3. Too many parameters easily lead to overfitting and are hard to train.

If we increase the kernel count of the last conv layer to 1024, the parameter count of this layer would be:

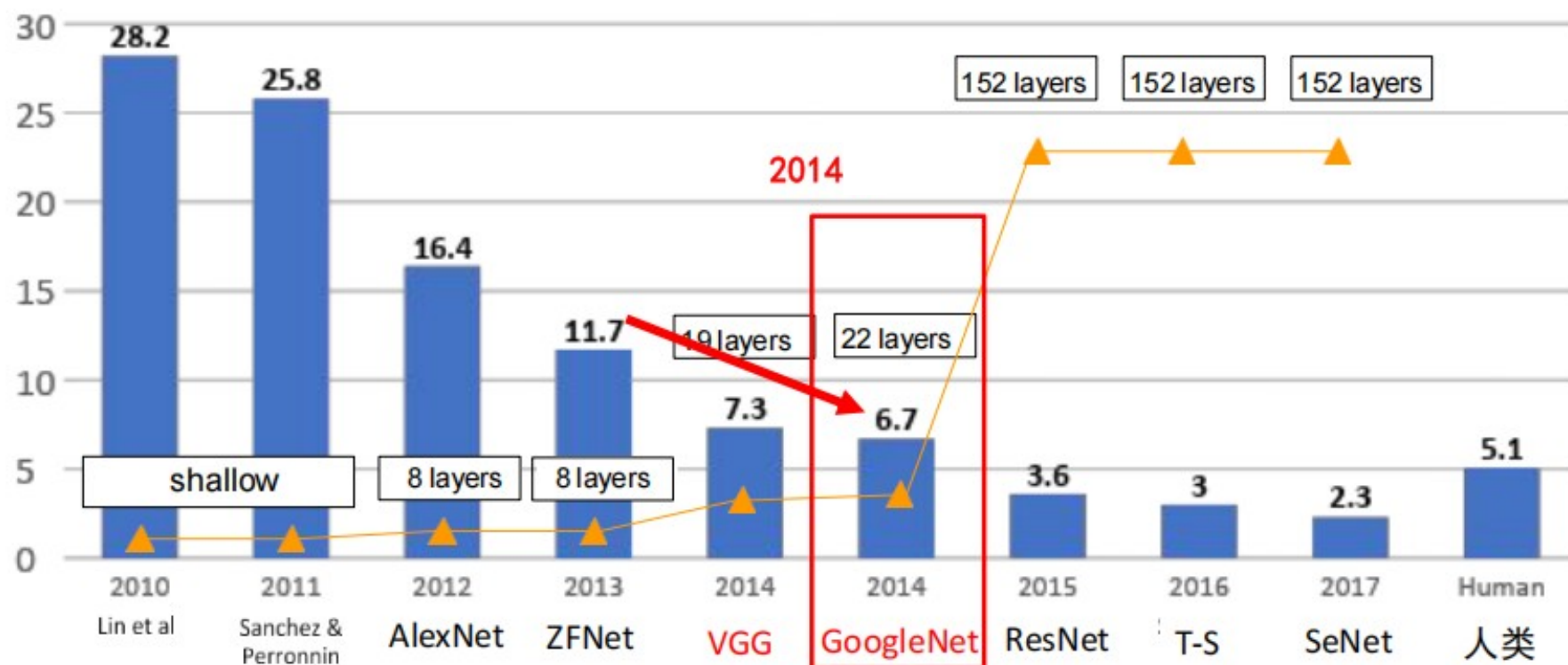
$$7 \times 7 \times 1024 \times 4096 = 205,520,896$$

$$\approx 200 \text{ M}$$

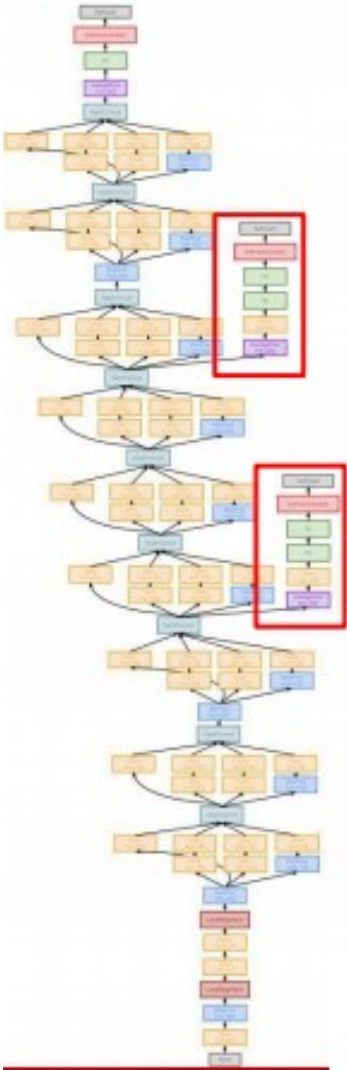


GoogleNet

ImageNet Challenge Winners Over the Years



Classic Network Analysis — GoogLeNet



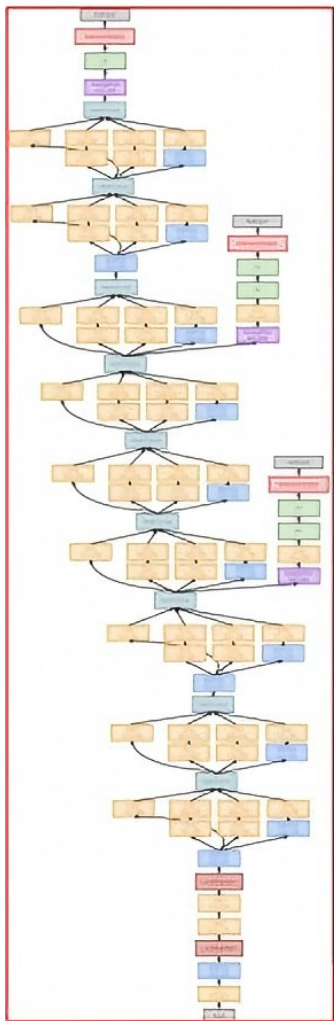
- GoogLeNet's Innovations:
- Proposed an Inception structure that retains more feature information from the input signal;
- Removed the first two fully-connected (FC) layers of AlexNet and replaced them with average pooling. GoogLeNet has only 5M parameters — about 12x fewer than AlexNet;
- Introduced auxiliary classifiers in the middle of the network to mitigate the vanishing-gradient problem during training.

Classic Network Analysis — GoogleNet

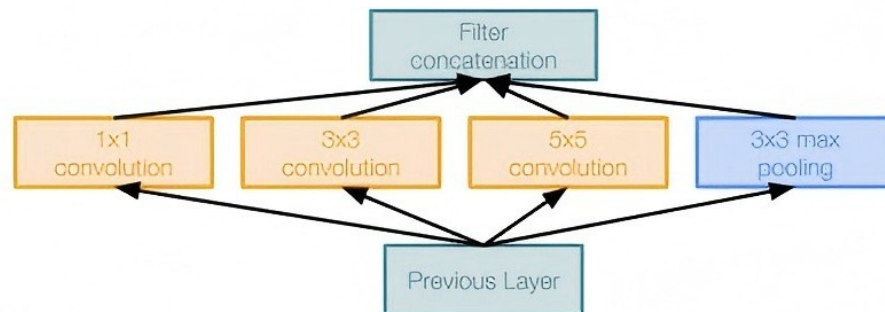


- Problem with the sequential structure (e.g., VGG) — later conv layers can only process the feature maps produced by the previous layer; if important information is dropped early, later layers cannot recover it.
- Solution — have each layer retain as much information from the input signal as possible.

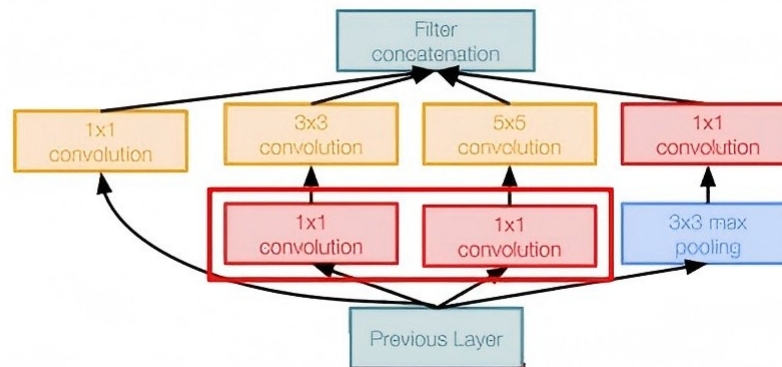
Classic Network Analysis — GoogleNet

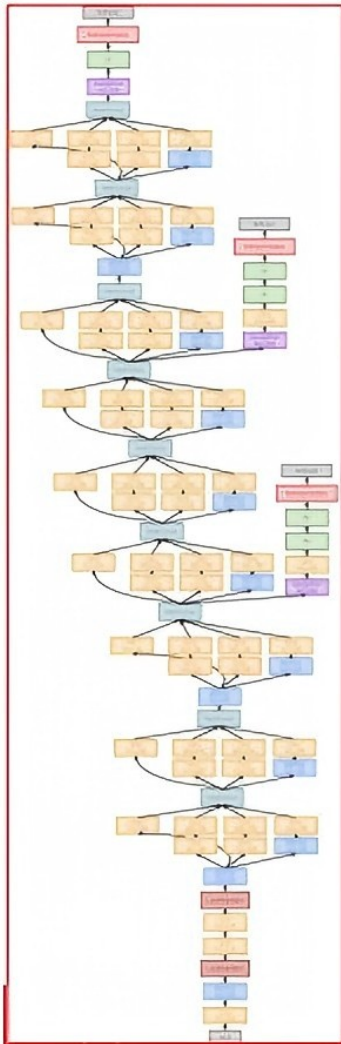


Inception

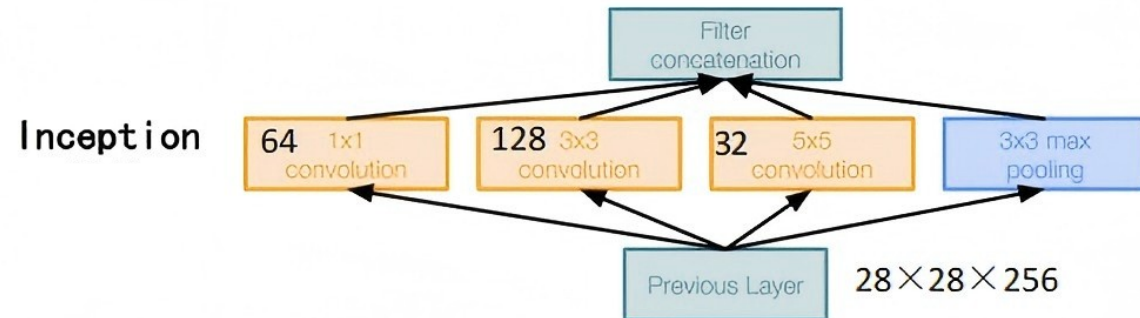


Inception V1





Classic Network Analysis — GoogleNet

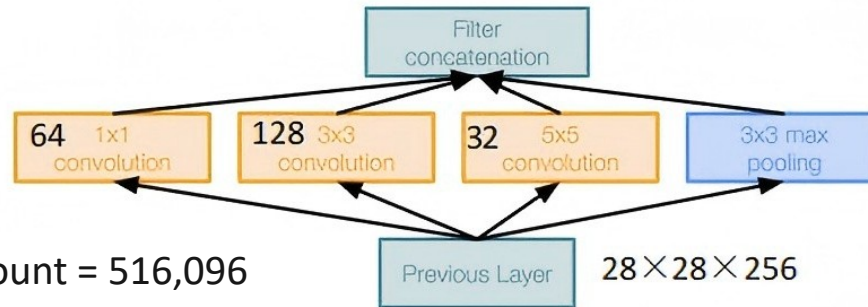


Parameter count = $1 \times 1 \times 256 \times 64 + 3 \times 3 \times 256 \times 128 + 5 \times 5 \times 256 \times 32 = 516,096$

Total computation = $28 \times 28 \times 516,096 = 404,619,264$

Classic Network Analysis — GoogleNet

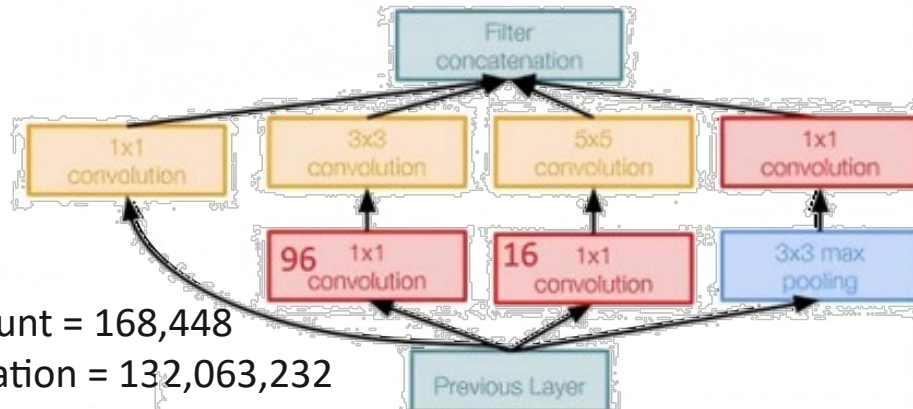
Inception



Parameter count = 516,096

Total computation = 404,619,264

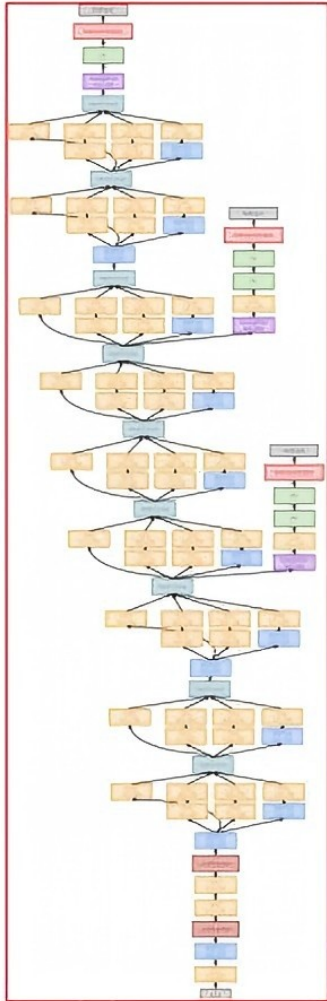
Inception V1



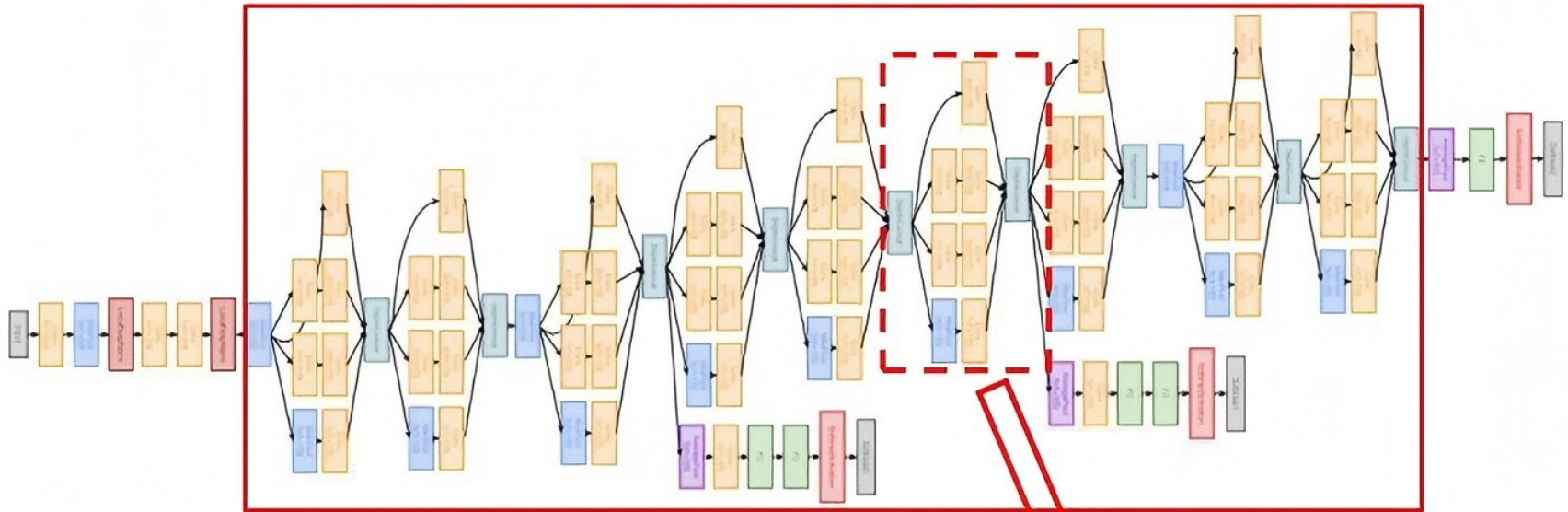
Parameter count = 168,448

Total computation = 132,063,232

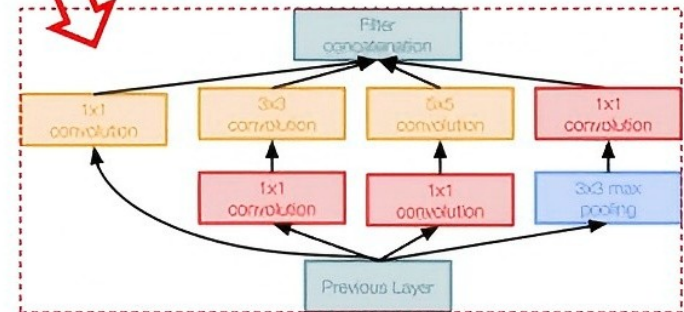
Deeper layers,
fewer
parameters,
higher
computational
efficiency, and
stronger non-
linear
expressive
power



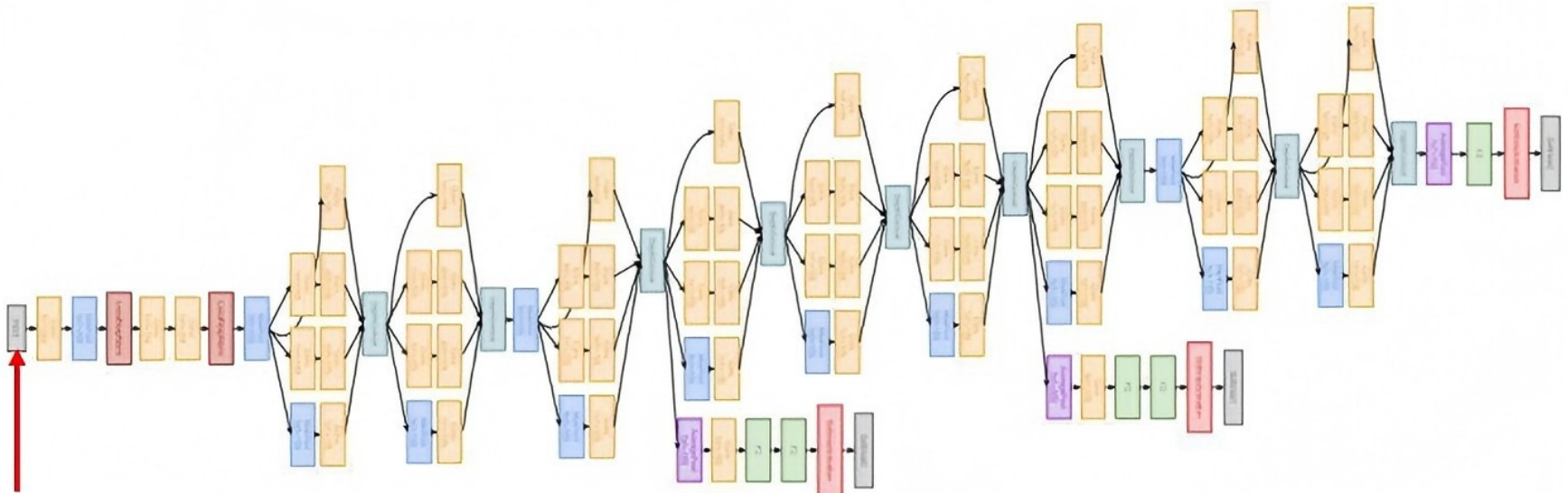
Classic Network Analysis — GoogleNet



Stacked 9 Inception V1 modules

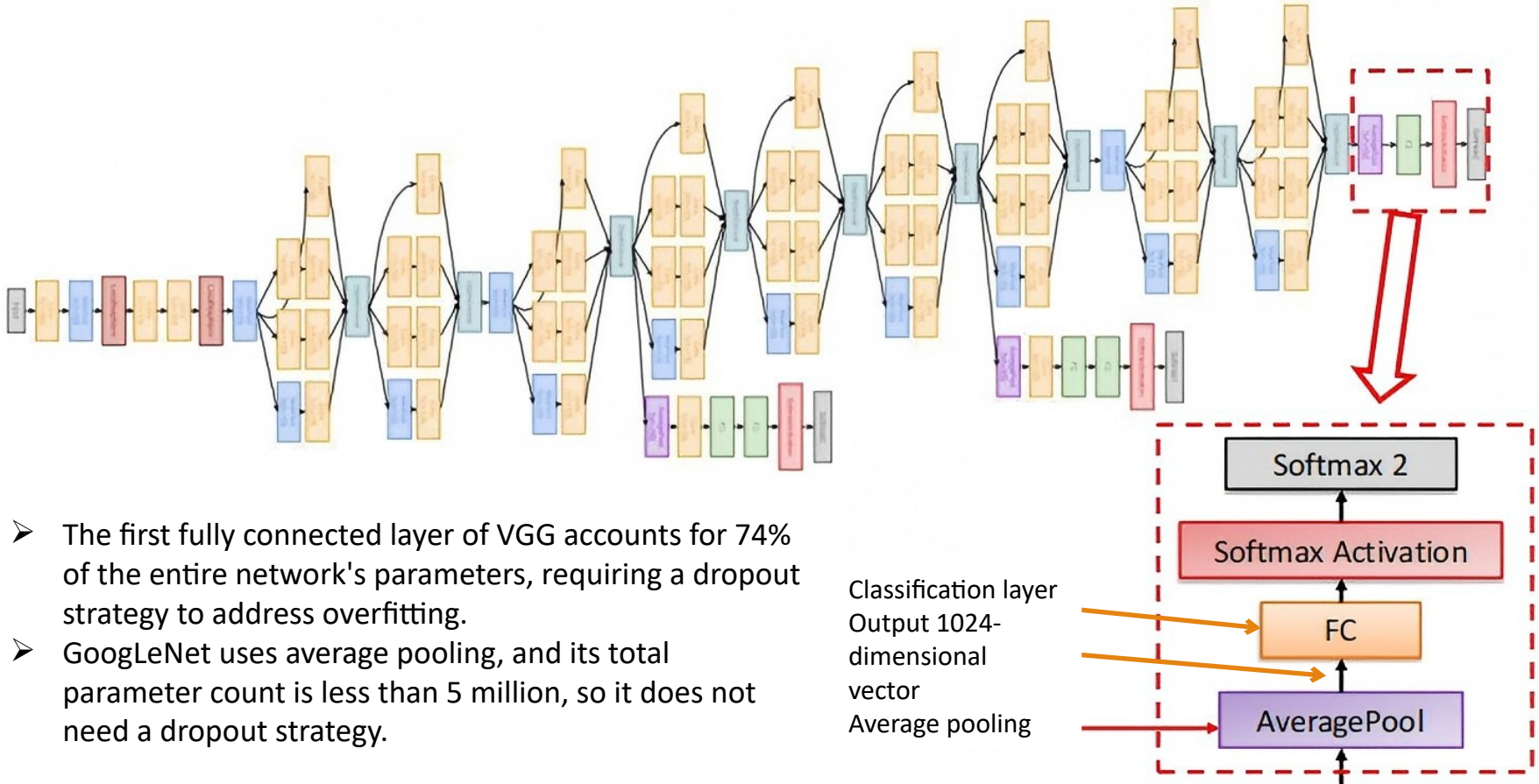


Classic Network Analysis — GoogleNet

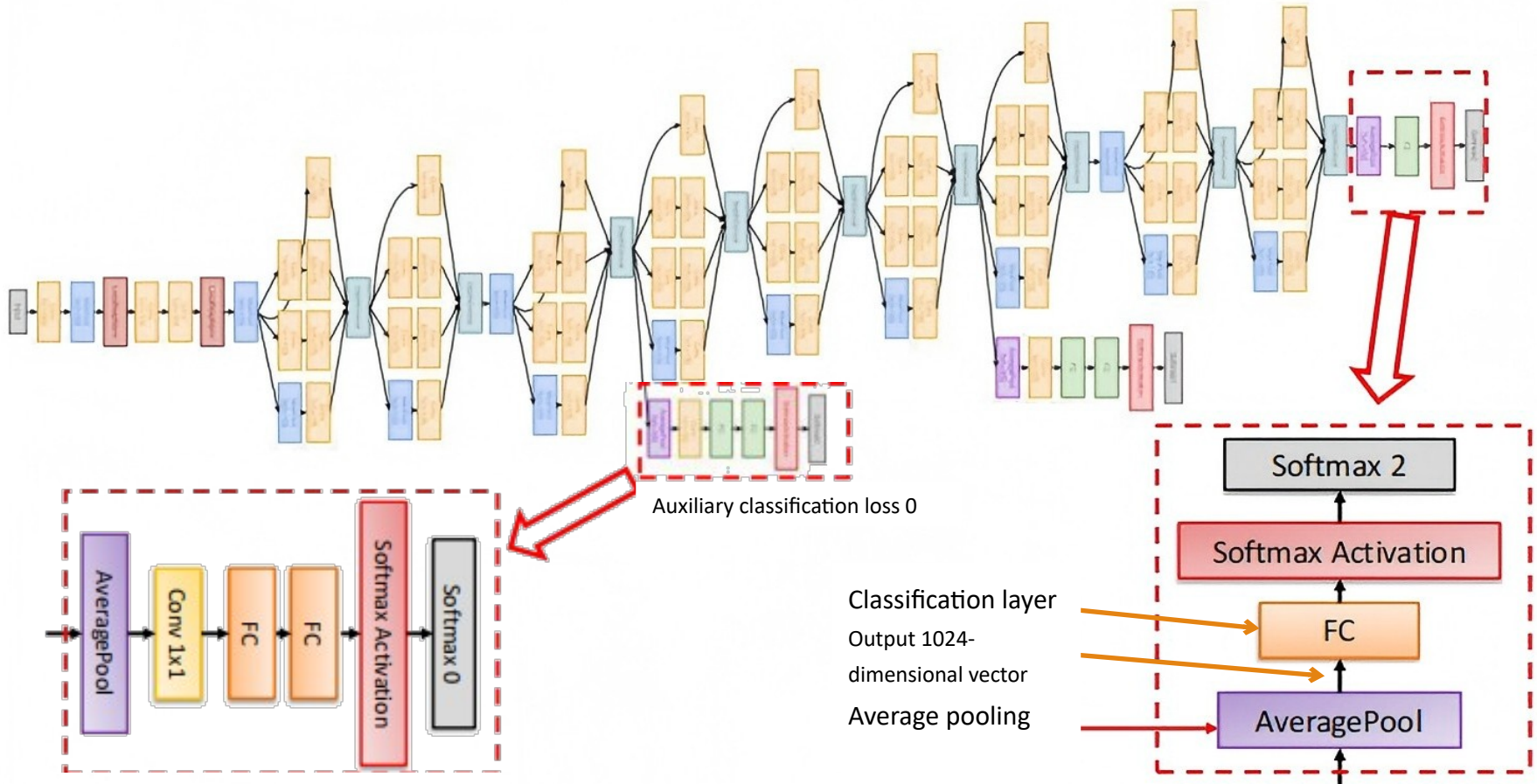


Input: Subtracted
mean
224×224
color image

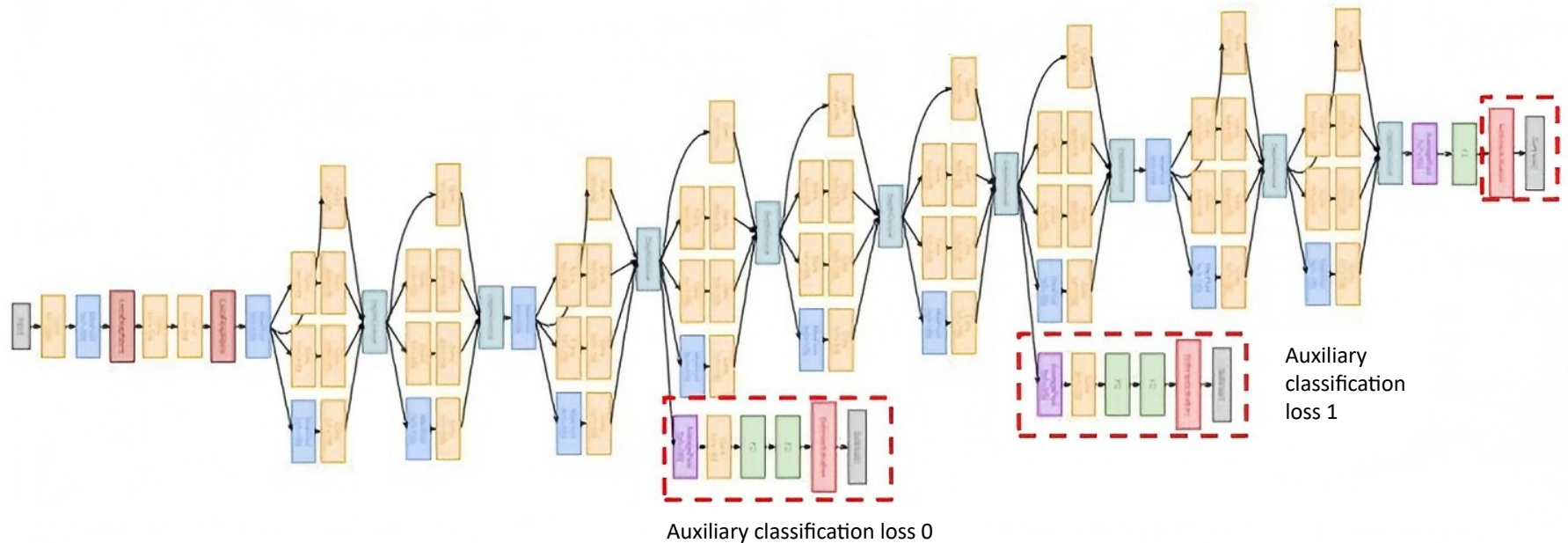
Classic Network Analysis — GoogLeNet



Classic Network Analysis — GoogleNet

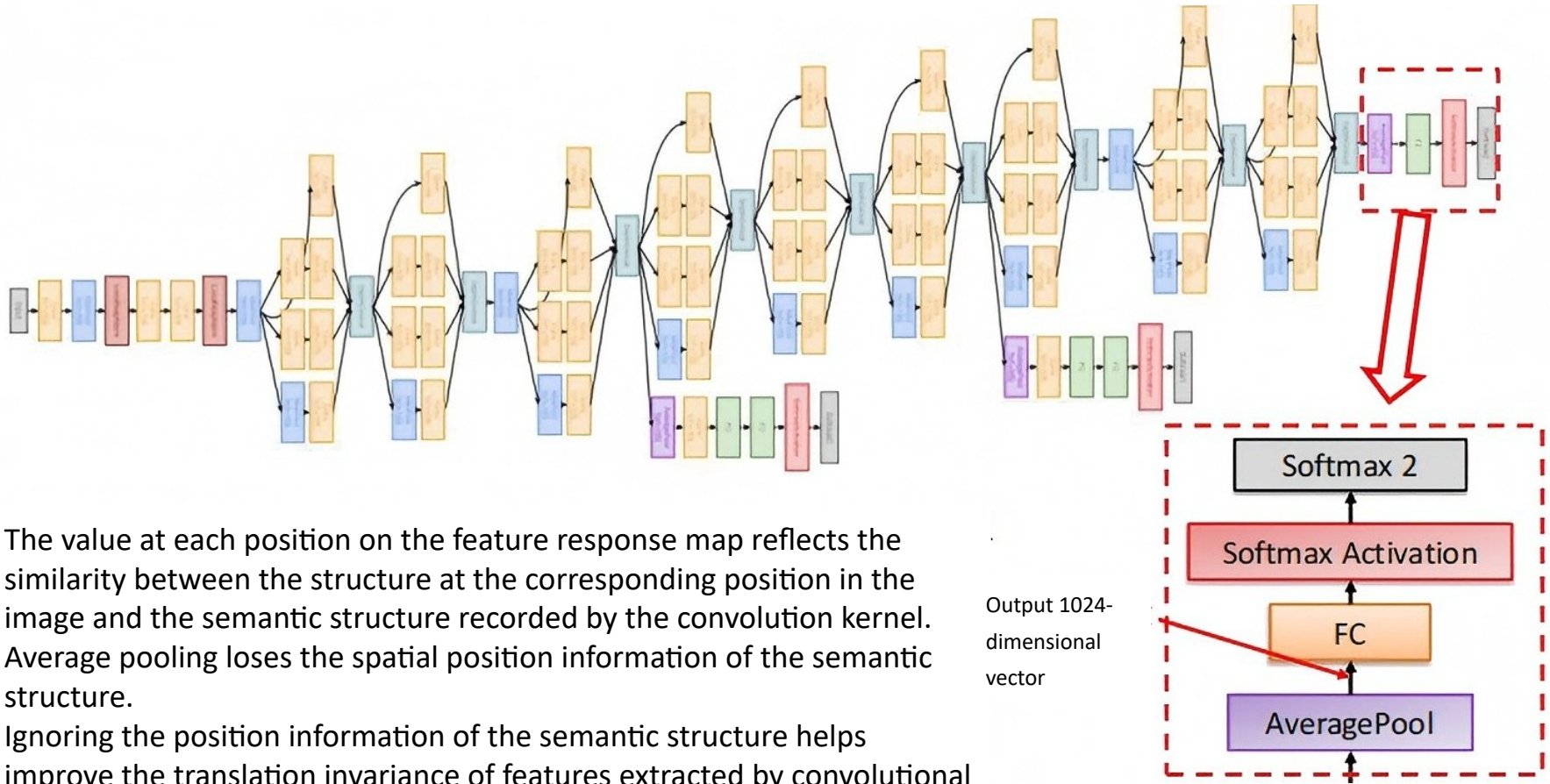


Classic Network Analysis — GoogleNet



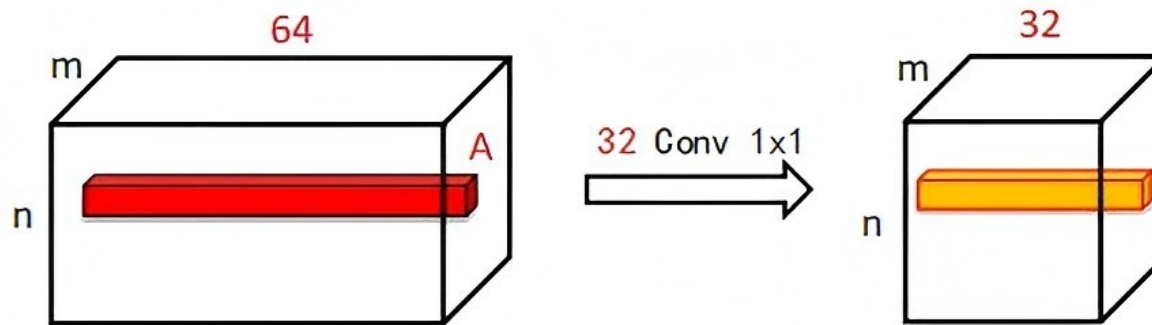
- Reason - Although ReLU units can partially solve the vanishing gradient problem, they cannot fully address the difficulty of training deep networks. Layers far from the output are not trained as well as layers closer to the output.
- Result - Enable the features learned by lower convolutional layers to also have strong discriminative ability, thereby allowing the network to be better trained. Moreover, when lower convolutional layers learn good features, it can also accelerate the convergence of the entire network.
- Network inference - Only use the final output of the network as the prediction result, ignoring the output of the auxiliary classifiers.

Question 1: What is the difference between average pooling vectorization and direct flattening vectorization?



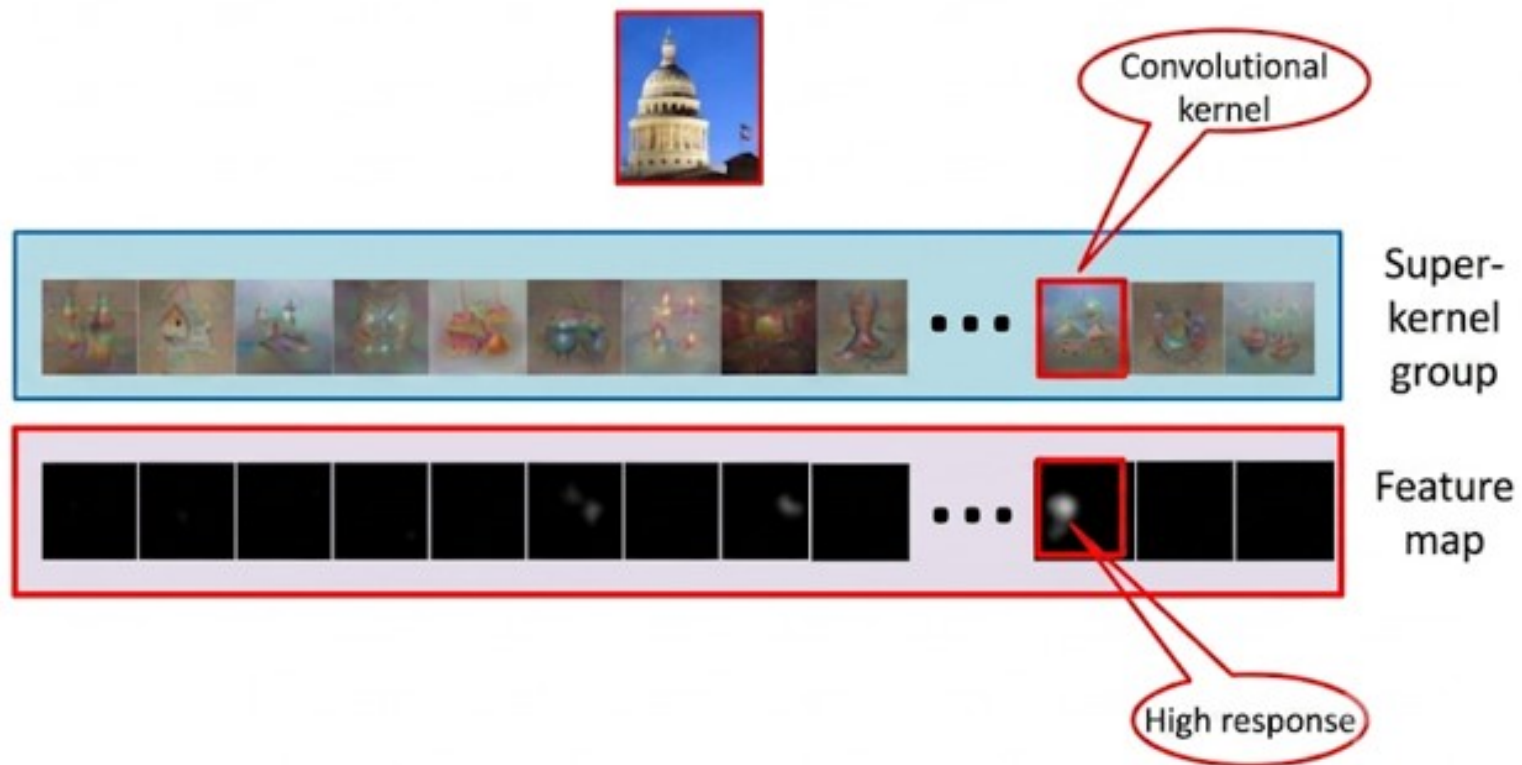
- The value at each position on the feature response map reflects the similarity between the structure at the corresponding position in the image and the semantic structure recorded by the convolution kernel.
- Average pooling loses the spatial position information of the semantic structure.
- Ignoring the position information of the semantic structure helps improve the translation invariance of features extracted by convolutional layers.

Question 2: Does using 1×1 convolution for compression lose information?



- Will this compression lose information?
- The 64-dimensional vector at position A is a very sparse vector, and nonlinear compression using 1×1 convolution usually does not lose information.

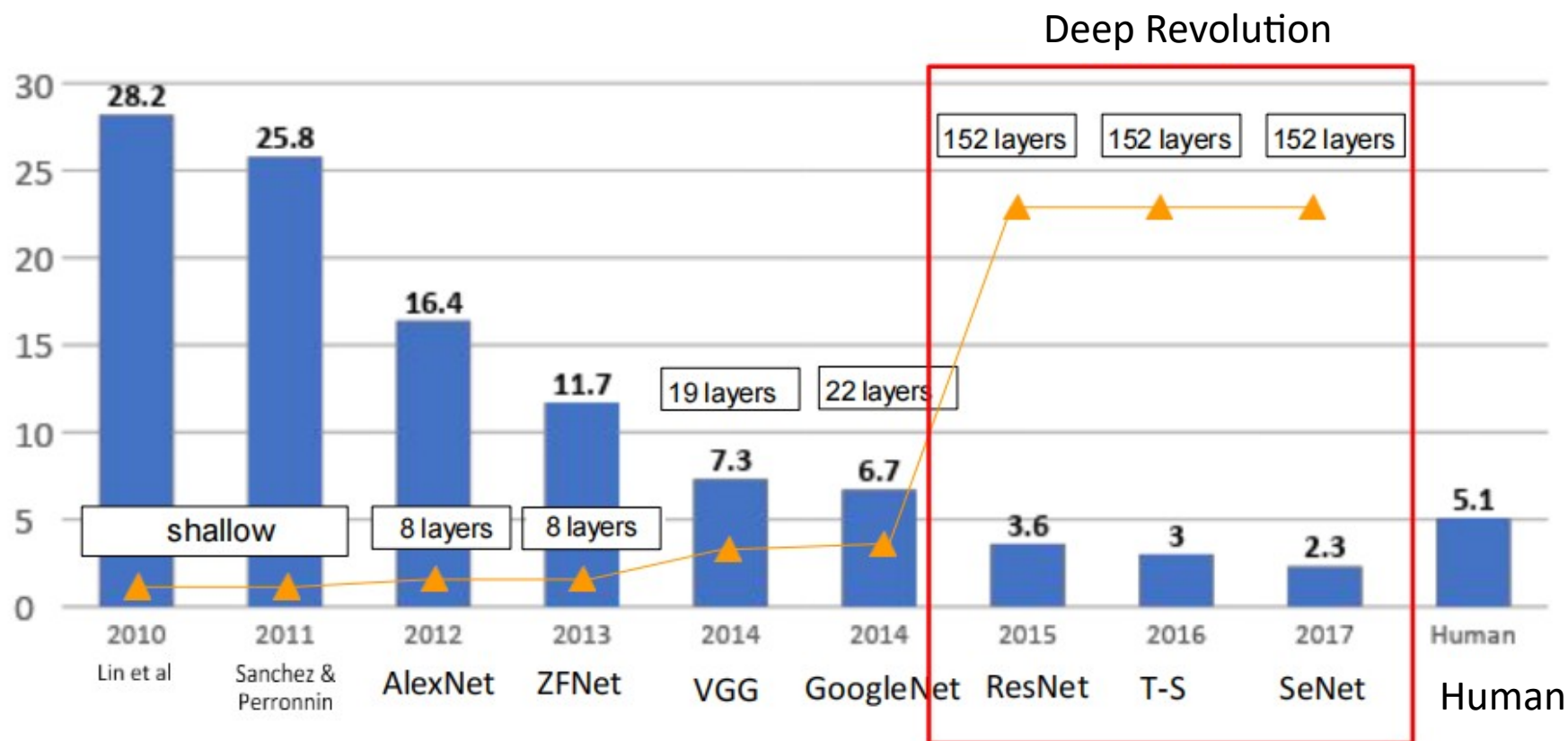
Question 2: Does using 1×1 convolution for compression lose information?





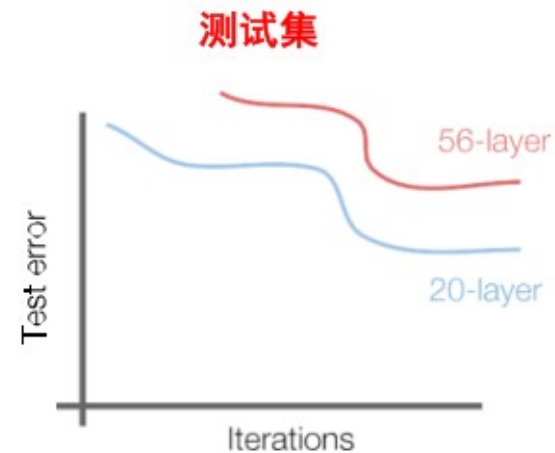
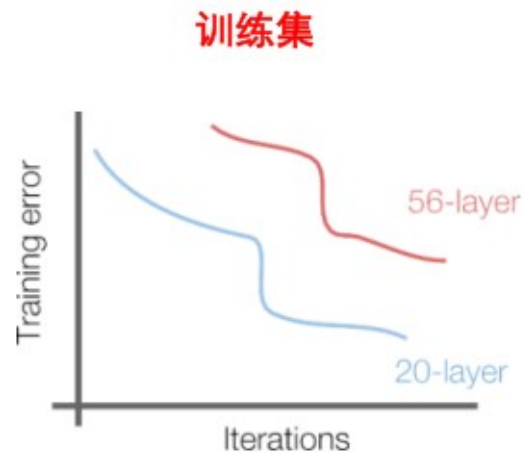
ResNet

ImageNet Challenge Winners Over the Years



Classic Network Analysis — ResNet

Experiment: What happens when deeper layers are continuously added to a "basic" convolutional neural network?



Why is this?

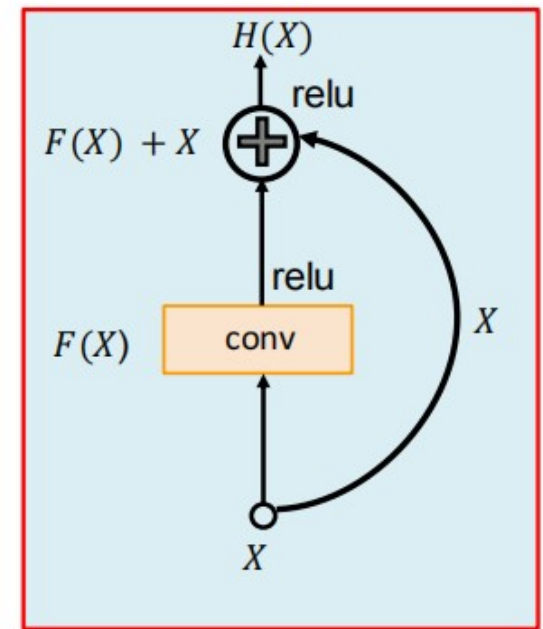
Reason: During the training process, the forward and backward information flow of the network is not smooth, and the network has not been fully trained.

Guess: Increasing the number of network layers causes overfitting, leading to higher error rates.

Classic Network Analysis —ResNet

ResNet has the following contributions:

- Proposed a residual module, by stacking which neural networks of arbitrary depth can be constructed without the "degradation" phenomenon.
- Proposed batch normalization to combat gradient vanishing, a method that reduces the dependence of the network training process on weight initialization;
- Proposed an initialization method for the ReLU activation function;



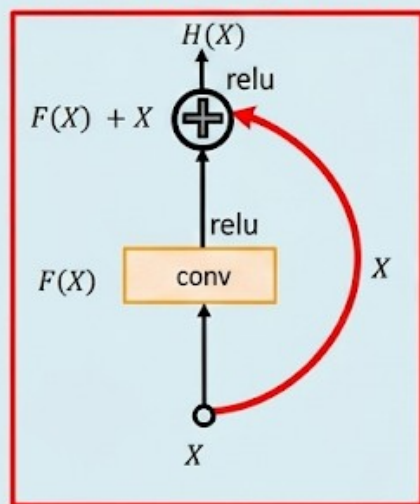
Residual Module

Classical Network Analysis — ResNet

The researchers considered this question:

After a shallow network learns an effective classification pattern, how to stack new layers to build a deeper network such that, even if it cannot improve the performance of the shallow network, the deeper network should not decrease performance.

Identity Mapping Structure



Residual Block

Solution: Residual Block

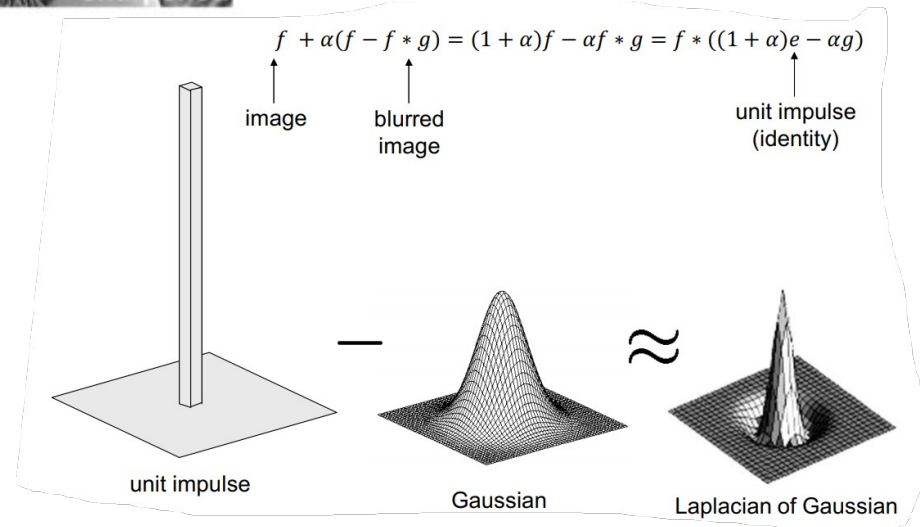
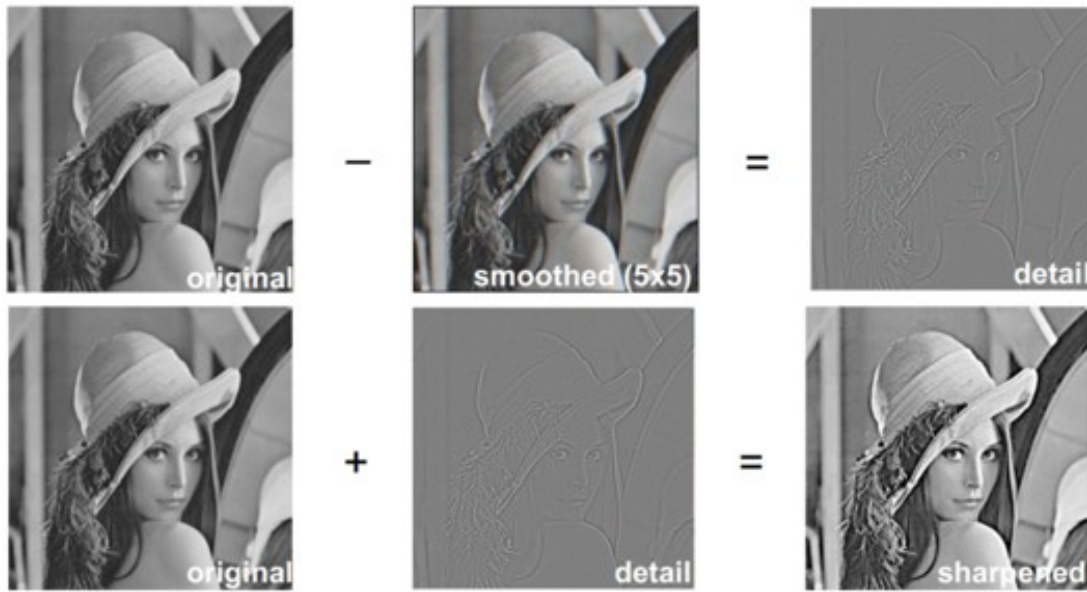
Assuming the transformation learned by the convolutional layers is $F(X)$, and the output, the residual structure is $H(X)$, we have:

$$H(X) = F(X) + X$$

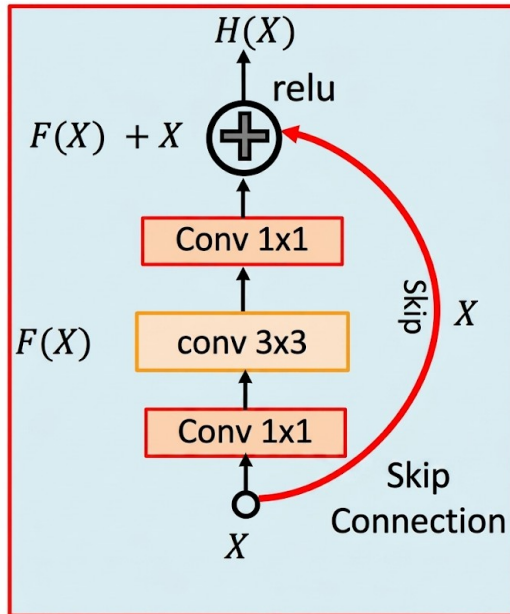
$$F(X) = H(X) - X$$

↑ ↑ ↑
Residual Output Input

Residual = Output - Input



Classic Network Analysis —ResNet



'Bottleneck' Structural
Residual Module

Regarding the residual structure:

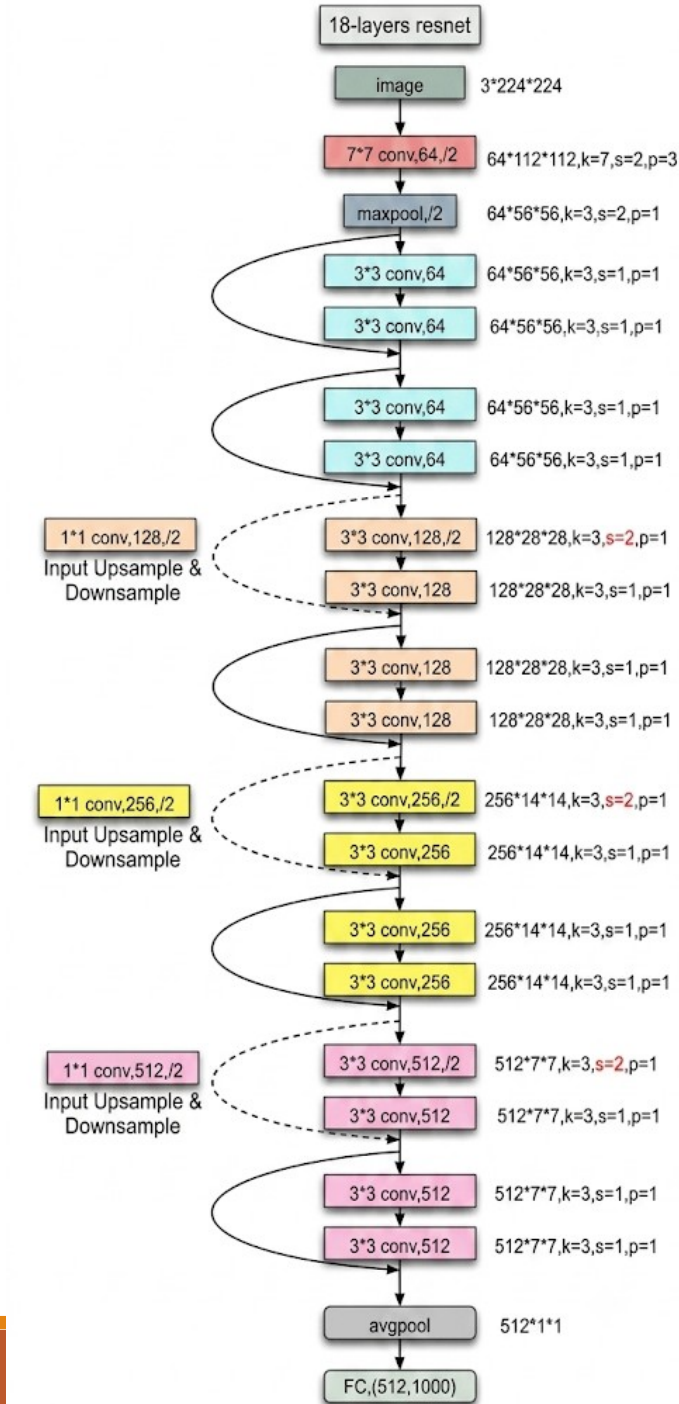
1. The residual structure can avoid the information loss problem that exists in ordinary convolutional layer stacking, ensuring the smooth flow of forward information.
2. The residual structure can address the vanishing gradient problem during backpropagation, ensuring the smooth flow of backward gradients.

Classic Network Analysis —ResNet

layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
conv2_x	56×56	3×3 max pool, stride 2				
		$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$
conv3_x	28×28	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 8$
conv4_x	14×14	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 23$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 36$
conv5_x	7×7	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

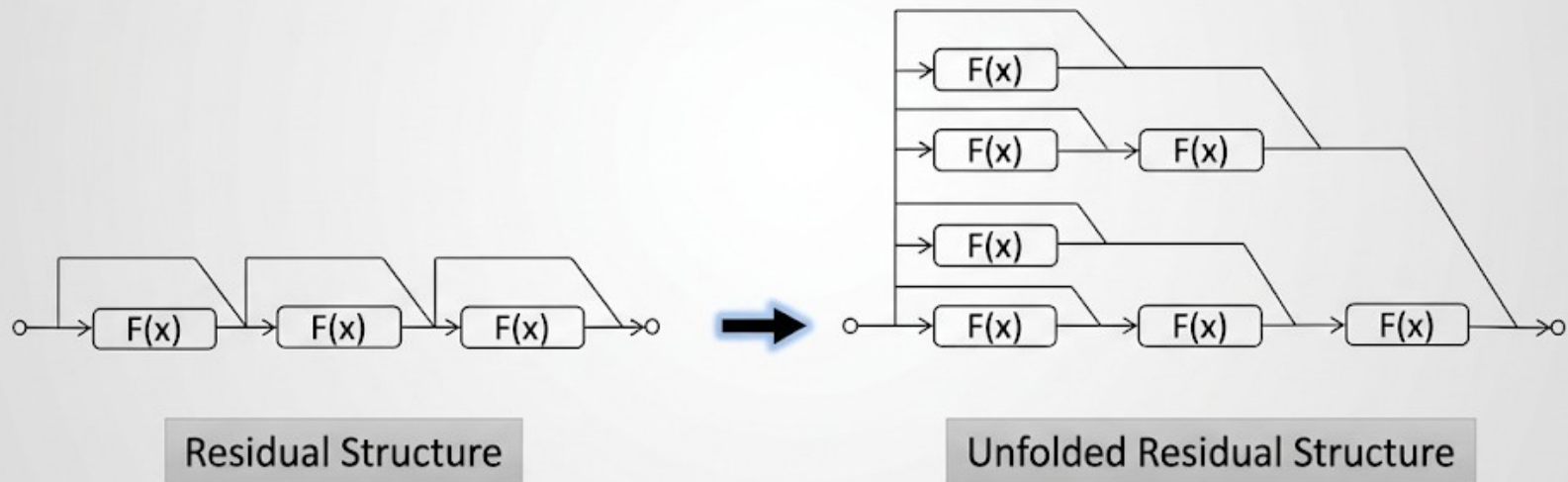
Other key points:

- Proposed a batch normalization strategy to reduce the dependence of network training on weight initialization.
- Proposed an initialization method for ReLU nonlinear units.



Question: Why do residual networks perform so well?

A typical explanation: Residual networks can be viewed as an ensemble model!



Conclusion: Residual networks are an ensemble model, which is an important feature and a reason for their high performance! ✨

Remove several layers of the network, and the residual network can still work!

Chapter Summary

- Introduced five classic convolutional neural networks: AlexNet, ZFNet, VGG, GoogLeNet, and ResNet
- Residual networks and InceptionV4 are widely recognized as the two best classification models for generalization performance
- Models for special application environments: SqueezeNet for limited storage resources, and MobileNet and ShuffleNet for limited computational resources